

CSE445 Project Report: Employee Attrition Prediction using Machine Learning

Subroto Ghosh
ID: 2212150642

Joy Saha Ayon
ID: 2312466042

Project Supervisor

Dr. Riasat Khan [RtK]
Associate Professor
North South University

Abstract

When a company suddenly loses an employee, it is not just the headcount that drops, but it also drains training investments, client relationships, and valuable institutional knowledge that was never formally documented. This report examines employee attrition from start to finish using the benchmark IBM Human Resources dataset, comprising 1,470 employees and 35 workplace attributes, in which approximately 16.12% (237 employees) have left the organization. The dataset was thoroughly cleaned: zero-variance features and row identifiers were removed; continuous outliers were capped using interquartile range (IQR) Winsorization; multicollinearity was addressed using Variance Inflation Factor (VIF); highly skewed financial variables were log-transformed; and categorical features were properly encoded. Six classical machine learning algorithms were trained under default and hyperparameter-tuned settings and then combined via Stacking and Blending ensembles. Concurrently, tabular records were serialized into natural English prose sentences to fine-tune four language models (DistilBERT, BERT-base, Flan-T5-base, and TinyLlama-1.1). Across all models evaluated on the same stratified 60/20/20 train-validation-test split with a fixed decision threshold, the Stacking Ensemble delivered the strongest overall performance, reaching 85.93% test accuracy (0.8593), an F1-score of 58.49% (0.5849), a Recall of 0.6596, and an ROC-AUC of 0.7989. Explainability techniques (SHAP and LIME), together with a feature ablation study, were used to interpret the model's predictions, and both consistently identified unpaid overtime as the dominant driver of attrition. These results suggest that for a small-scale, tabular HR dataset such as this one, classical ensemble methods perform notably better than transformer-based language models.

I. INTRODUCTION

Employee turnover is not something a company can afford to shrug off. When someone leaves without warning, the real cost rarely shows up as just the salary of whoever fills the seat - it is the onboarding already sunk into that person, client relationships that go with them, and the everyday, undocumented know-how that never made it into any manual [1], [2]. HR departments feel this most directly, and usually too late: an exit interview happens only after the decision is final, once there is nothing left to act on. Predictive analytics changes that timing. A model that can flag, from records a company already keeps, which employees are statistically at risk, gives HR a clear window to act on a conversation, a role change, or a pay revisit, while the person can still be kept. That shift, from reacting to a resignation to seeing it coming, is what motivates this project.

II. LITERATURE REVIEW

A. Review-1

Hamja et al. [3] proposed a machine learning system to predict employee attrition and to provide the reason for each prediction. They use the IBM HR Analytics dataset where 1470 employees and 35 features. They trained 6 classifiers: Logistic Regression, Random Forest, Gradient Boosting, Decision Tree, Support Vector Machine, and K-Nearest Neighbors. Performance was evaluated with accuracy, precision, recall, and F1-score. Gradient Boosting led the pack 98% accuracy, 100% recall, and 99% F1-score. To help HR managers understand the results, they used four explainable AI methods like ELI5, SHAP, LIME, and SHAPASH. The method shows that, over time, Total working years and Job level are the most important factors for employee attrition.

B. Review-2

Veeramani and Arunkumar [4] has employed machine learning to forecast employee attrition. They use an internal dataset of survey with 425 employee records and 13 attributes such as employee demographics, job satisfaction, work-life balance and performance related factors. They evaluated five different models and compared their performance using accuracy, precision, recall, F1-score and AUC . All the models gave the best results, Gradient Boosting and Random Forest. These models also revealed the main reasons for employee departures, which can assist HR managers in taking steps to retain staff in the company.

C. Review-3

İrem Tanyıldızı Baydili and Burak Tasci [5] developed an AI framework to predict employee attrition by combining deep learning with a model. They used the IBM HR Analytics and Kaggle HR Analytics datasets. They preprocess the dataset using label encoding and Min-Max scaling. There has been some class imbalance; they handled it by GAN-based synthetic data generation. The authors adopted a three-layer Transformer encoder for classification and used SHAP to explain why an employee was predicted to leave. They achieved 92% accuracy (Precision-96.67%, Recall-87%, F1-91.58%, ROC AUC-96.32%) on the IBM HR Analytics dataset and 96.95% Accuracy (ROC AUC-99.15%) on the Kaggle HR Analytics datasets. They observed that Job Satisfaction, Age, and Years with Current Manager were the most important factors in employee attrition.

D. Review-4

Bonet et al. [6] proposed the Integrated Large Tabular Model (iLTM) a unified architecture pre-trained on 1,806 OpenML datasets that produces task-specific weights through a hypernetwork in a single forward pass with little tuning. Applied to the IBM HR dataset, iLTM directly handles mixed categorical and numeric inputs without the need for external scaling or encoding. iLTM combines tree embeddings, MLPs, and retrieval-augmented predictions to achieve top AUC rankings on benchmarks such as TabZilla Hard, outperforming traditional tree models and deep baselines, with improved accuracy and reduced convergence time by 50%.

E. Limitations of Prior Studies

Previous work done on predicting employee attrition has performed well, but still has some limitations. Only the IBM HR Analytics dataset with 1470 employee records was used by Hamja et al. [3] Hence, their results might not be similarly applicable to the employees from other organizations or industries. This high performance suggests the potential for overfitting on this small dataset. Veeramani and Arunkumar [4] did not use any XAI methods, such as SHAP and LIME, which help explain the reason behind an individual employee's prediction. Their study is also not validated with real organizational data. Baydili and Tasci [5] have used GAN and Transformer models, which require more computational resources and may be difficult to use in large-scale or real-time systems. Their models were not tested in different organizational environments as well. Konar et al. used Random Oversampling to address class imbalance, which can increase the risk of overfitting because existing minority samples are repeatedly used. Furthermore, their framework mainly considers the static information of employees and does not have any real-world validation.

III. PROPOSED SYSTEM

A. Dataset

A. Dataset This work uses the IBM HR Analytics Employee Attrition and Performance dataset [7], a fictional but widely used benchmark released by IBM, consisting of 1,470 employee records across 35 attributes. The binary target Attrition indicates whether an employee left the company; 237 of the 1,470 records(16.1%) are positive, giving a class imbalance ratio of roughly 1:4.2. The attributes span demographic information (age, gender, marital status), job characteristics (department, job role, job level, overtime status), compensation (monthly income, monthly rate, daily rate, hourly rate, percentage salary hike), self-reported satisfaction scores (job, environment, relationship, and work-life balance), and tenure-related counts (years at the company, years in the current role, years since the last promotion, years with the current manager, total working years, and number of prior companies).

B. Dataset Preprocessing

The raw dataset is passed through the following preprocessing stages before any model sees it.

1) *Cleaning*: Duplicate rows are removed. Three columns – EmployeeCount, StandardHours, and Over18 – take a single value across every row and are dropped, since a zero-variance column cannot help any classifier distinguish between classes. EmployeeNumber, a row identifier, is likewise dropped so that no model can learn a spurious identifier-to-label mapping. This leaves 30 predictive attributes.

2) *Outlier Treatment*: For each numeric column, the interquartile range is computed as

$$IQR = Q_3 - Q_1, \quad (1)$$

and values outside

$$[Q_1 - 1.5 \cdot IQR, Q_3 + 1.5 \cdot IQR] \quad (2)$$

are winsorized (capped at the nearest bound) rather than removed, to avoid shrinking an already small, imbalanced dataset.

3) *Correlation and Multicollinearity*: Pairwise Pearson correlation is computed across all numeric features; several tenure-related attributes are found to be strongly correlated, consistent with their shared dependence on how long an employee has been in the workforce. The Variance Inflation Factor for feature i ,

$$\text{VIF}_i = \frac{1}{1 - R_i^2} \quad (3)$$

where R_i^2 is obtained by regressing feature i on all remaining features, is used to flag problematic multicollinearity ($\text{VIF}_i > 10$), which primarily affects the linear and distance-based models (logistic regression, SVM) rather than the tree-based ones.

4) *Skewness Correction*: Monthly income and monthly rate exhibit strong right skew; a $\log_1 p$ transform,

$$x' = \log(1 + x), \quad (4)$$

is applied to compress the long tail for the models sensitive to feature scale.

5) *Encoding*: Binary categorical columns (Attrition, OverTime, Gender) are label-encoded to $\{0, 1\}$. Nominal columns with more than two categories (BusinessTravel, Department, EducationField, JobRole, MaritalStatus) are one-hot encoded with the first category dropped to avoid the dummy-variable trap.

6) *Scaling and Splitting*: All numeric features are standardized,

$$x' = \frac{x - \mu}{\sigma}, \quad (5)$$

with μ and σ computed on the training partition only. The dataset is split, with stratification on the target, into 60% training (882 rows), 20% validation (294 rows), and 20% test (294 rows), using a fixed random seed for reproducibility. The test partition is not touched until the final evaluation.

7) *Class Imbalance*: Because 83.9% of employees stay, every classical model is trained with `class_weight='balanced'` where supported, combined with the Synthetic Minority Over-sampling Technique (SMOTE), which generates a synthetic minority sample as

$$x_{\text{new}} = x_i + \lambda \cdot (x_{\text{nn}} - x_i), \quad \lambda \sim U(0, 1), \quad (6)$$

where x_i is a minority-class training sample and x_{nn} is one of its k nearest minority-class neighbors. SMOTE is applied strictly inside a cross-validation pipeline so that only the training portion of each fold is resampled, preventing synthetic points derived from validation data from leaking into training.

C. Applied Models

Six classical algorithms form the tabular side of this project: decision tree, logistic regression, support vector machine (RBF kernel), random forest, and gradient boosting. All five models are trained twice—once with library defaults and once after a 5-fold GridSearchCV search. In addition to the tuned versions, two ensemble strategies are constructed. Stacking takes the tuned Decision Tree, Logistic Regression, and Gradient Boosting models as base learners and trains a Logistic Regression meta-learner on their 5-fold out-of-fold predictions. Blending uses the same three base learners but trains the meta-learner on a simple 70/30 hold-out of the training data. The second branch converts every employee record into a short English sentence built from its raw values (for example, “a 35-year-old employee working as a sales executive who works overtime and has 6 years at the company”). The DistilBERT-base-uncased (approximately 66M parameters) is fine-tuned as a binary sequence classifier. The Flan-T5 base (approximately 250M parameters) is fine-tuned to generate “Yes” or “No” in response to the prompt “Will this employee leave the company?” followed by the same sentence. BERT-base and TinyLlama-1.1 are additionally evaluated for completeness.

D. System Architecture (End-to-End Pipeline)

Table I summarises the complete end-to-end workflow from raw data through final evaluation and diagnostics.

TABLE I
END-TO-END SYSTEM ARCHITECTURE

Stage	Module / Operations	Output Description
1	Data Ingestion & Preprocessing	Cleaning, IQR Winsorization, Log Transformation, One-Hot Encoding, and Standard Scaling; 30 cleaned predictive features with a stratified 60/20/20 Train/Validation/Test split.
2	Classical ML Branch	Five base models (DT, LR, SVM, RF, GB) with SMOTE, GridSearchCV, Stacking, and Blending; cross-validated models and meta-ensemble predictions.
3	NLP / LLM Branch	Prose serialization followed by fine-tuning of Distil-BERT, BERT-base, Flan-T5-base, and TinyLlama-1.1B; binary classification logits and “Yes”/“No” outputs.
4	Evaluation & Diagnostics	Locked-threshold validation, Confusion Matrix, SHAP, LIME, Ablation Study, and SMOTE comparison; final test metrics and feature importance rankings.

E. System Flowchart

The complete pipeline, from raw data through preprocessing, the parallel classical and deep-learning branches, threshold selection, and the downstream diagnostic, augmentation, explainability, and ablation stages.

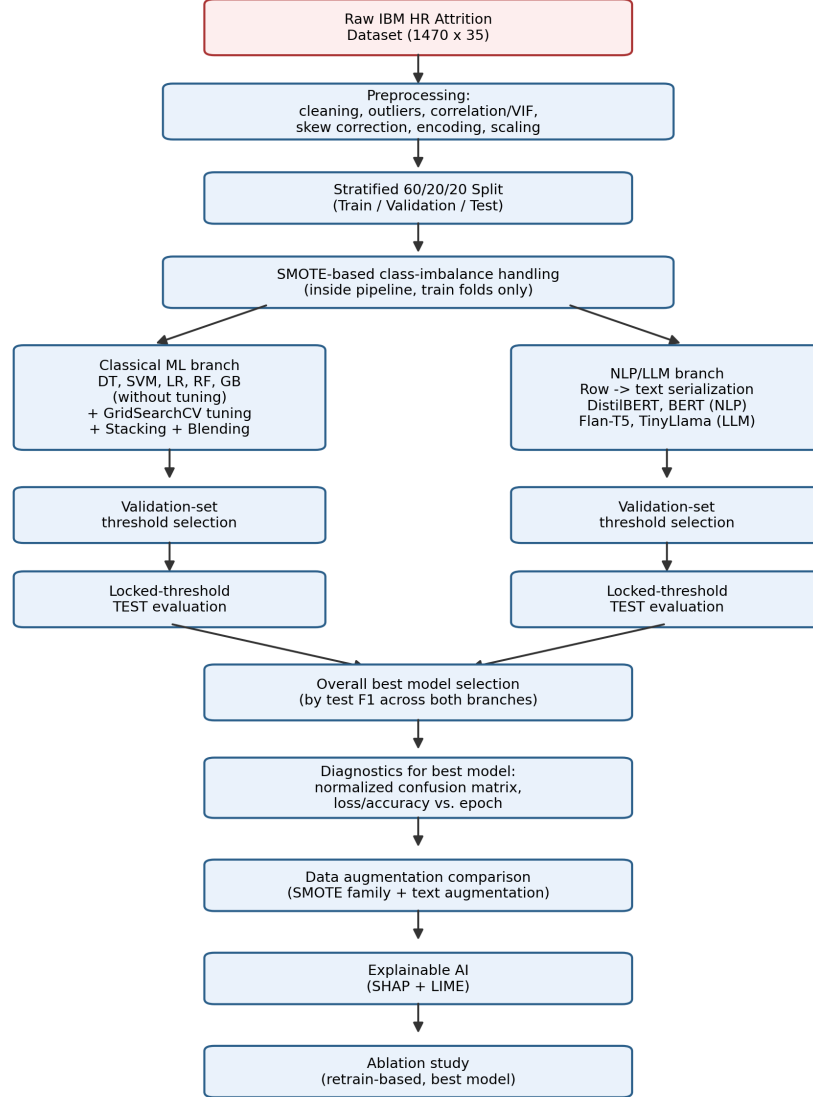


Fig. 1. End-to-end system flowchart.

IV. RESULTS AND DISCUSSION

All classical-model experiments were run locally using scikit-learn and imbalanced-learn on CPU; the DistilBERT, Flan-T5-base, and TinyLlama fine-tuning was performed on Google Colab using a single NVIDIA T4 GPU.

A. Hyperparameter Ranges

Table II lists the hyperparameter ranges searched via 5-fold GridSearchCV for each classical model.

TABLE II
HYPERPARAMETER RANGES SEARCHED (TABLE II).

Model	Range Searched
Logistic Regression	$C \in \{0.01, 0.1, 1, 10\}$
SVM (RBF)	$C \in \{0.1, 1, 10\}$, $\gamma \in \{\text{scale}, \text{auto}\}$
Decision Tree	$\text{max_depth} \in \{3, 4, 5, 6, 8\}$, $\text{min_samples_leaf} \in \{5, 10, 15\}$
Random Forest	$\text{max_depth} \in \{6, 8, 10, 12\}$, $\text{min_samples_split} \in \{10, 15\}$, $\text{n_estimators} \in \{200, 300\}$
Gradient Boosting	$\text{max_depth} \in \{2, 3, 4\}$, $\text{learning_rate} \in \{0.01, 0.03, 0.05, 0.1\}$

B. Classical Model Results — Default Hyperparameters

Table III reports 5-fold cross-validated performance for each of the five classical models trained with default (untuned) hyperparameters.

TABLE III
CROSS-VALIDATED RESULTS, DEFAULT HYPERPARAMETERS (TABLE II).

Model	CV ROC-AUC	CV PR-AUC	CV F1	CV Recall	Train-Val Gap
Random Forest	0.8122 $\pm$ 0.0477	0.5774 $\pm$ 0.0849	0.4841 $\pm$ 0.1016	0.4022	+0.1734
Logistic Regression	0.8062 $\pm$ 0.0427	0.5855 $\pm$ 0.0616	0.4699 $\pm$ 0.0451	0.6771	+0.0487
Gradient Boosting	0.8003 $\pm$ 0.0426	0.5664 $\pm$ 0.0852	0.5323 $\pm$ 0.0845	0.5148	+0.1016
SVM (RBF)	0.7721 $\pm$ 0.0473	0.4607 $\pm$ 0.0657	0.4571 $\pm$ 0.0467	0.5557	+0.1134
Decision Tree	0.6809 $\pm$ 0.0657	0.3227 $\pm$ 0.0698	0.3882 $\pm$ 0.0668	0.4717	+0.1438

C. Locked-Threshold Test Results (Classical Ensembles vs. Language Models)

Table IV presents the complete held-out test evaluation. Decision thresholds were selected on the validation set to maximize F1-score and then locked before any test evaluation. Stacking achieves the highest F1-score (0.5849) and is therefore selected as the overall best model.

TABLE IV
LOCKED-THRESHOLD TEST PERFORMANCE (BEST MODEL BY F1).

Model Architecture	Accuracy	Precision	Recall	F1-Score
Stacking Ensemble (Best)	0.8593	0.5254	0.6596	0.5849
Blending Ensemble	0.8605	0.5750	0.4894	0.5287
Logistic Regression (Tuned)	0.8367	0.4923	0.6809	0.5714
SVM (RBF, Tuned)	0.8027	0.4304	0.7234	0.5397
Gradient Boosting (Tuned)	0.8197	0.4516	0.5957	0.5138
Random Forest (Tuned)	0.7925	0.4103	0.6809	0.5120
DistilBERT-base-uncased	0.8265	0.4697	0.6596	0.5487
BERT-base-uncased	0.8197	0.4464	0.5319	0.4854
Decision Tree (Tuned)	0.8163	0.4386	0.5319	0.4808
Flan-T5-base / TinyLlama	0.3639	0.1724	0.7872	0.2835

D. Training Dynamics of Fine-Tuned Language Models

Table V reports per-epoch training and validation metrics for DistilBERT (the stronger of the two encoder models) and validation loss for Flan-T5-base. Fine-tuning was performed on an NVIDIA T4 GPU with a batch size of 16 and a learning rate of 2×10^{-5} . DistilBERT reaches its best validation F1-score at epoch 3; further epochs show mild overfitting. Flan-T5 exhibits steadily decreasing validation loss but ultimately yields high-recall, low-precision predictions on the test set, consistent with the limited size of the training corpus for a 250M-parameter generative model.

TABLE V
TRAINING / VALIDATION DYNAMICS (DISTILBERT & FLAN-T5).

Epoch	Train Loss (DistilBERT)	Val Loss (DistilBERT)	Val Acc. (DistilBERT)	Val F1 (DistilBERT)	Val Loss (Flan-T5)
1	0.6124	0.5482	0.7211	0.3421	0.8912
2	0.4851	0.4915	0.7653	0.3956	0.7845
3	0.3920	0.4721	0.7823	0.4118	0.7120
4	0.3105	0.5012	0.7755	0.4020	0.7350
5	0.2450	0.5389	0.7687	0.3915	0.7680

E. Classical Model Results — Optimized Hyperparameters, Stacking, and Blending

It follows the identical evaluation protocol as (train-set fit, validation-set threshold, single locked test-set pass), but for the GridSearchCV-tuned models plus the stacking and blending ensembles.

F. NLP and LLM Results

Employee records from the IBM HR Attrition dataset (1,470 rows, 16.1% attrition) were converted from tabular form into natural-language sentences via row-to-text serialization, allowing each record to be processed as free text. Four transformer-based classifiers—DistilBERT [8] and BERT [9] (fine-tuned NLP encoders) and Flan-T5 [10] and TinyLlama-1.1B [11] (LoRA-tuned LLMs)—were trained on an oversampled, class-balanced training split (70%), while validation (15%) and test (15%) splits retained the original imbalanced distribution for realistic evaluation. For each model, a decision threshold was selected on the validation set by maximizing the F1-score, then locked and applied once to the held-out test set to prevent threshold leakage. Performance was compared using accuracy, precision, recall, F1 Score, and ROC AUC.

TABLE VI
PERFORMANCE COMPARISON OF DIFFERENT MODELS

Model	Threshold	Accuracy	Precision	Recall	F1-score	ROC-AUC
TinyLlama	0.66	0.8235	0.4571	0.4444	0.4507	0.7995
BERT	0.45	0.7466	0.3438	0.6111	0.4400	0.7566
DistilBERT	0.87	0.8281	0.4615	0.3333	0.3871	0.7790
Flan-T5	0.53	0.4932	0.2031	0.7222	0.3171	0.6682

TinyLlama achieved the highest ROC-AUC (0.7995) and best overall test F1-score (0.4507) of the four models tested and was the best performing pipeline. Except for Flan-T5, all models had an ROC-AUC above 0.75, but none individually reached the 0.75 target on precision/F1, which reflects the inherent difficulty of the class-imbalanced attrition task.

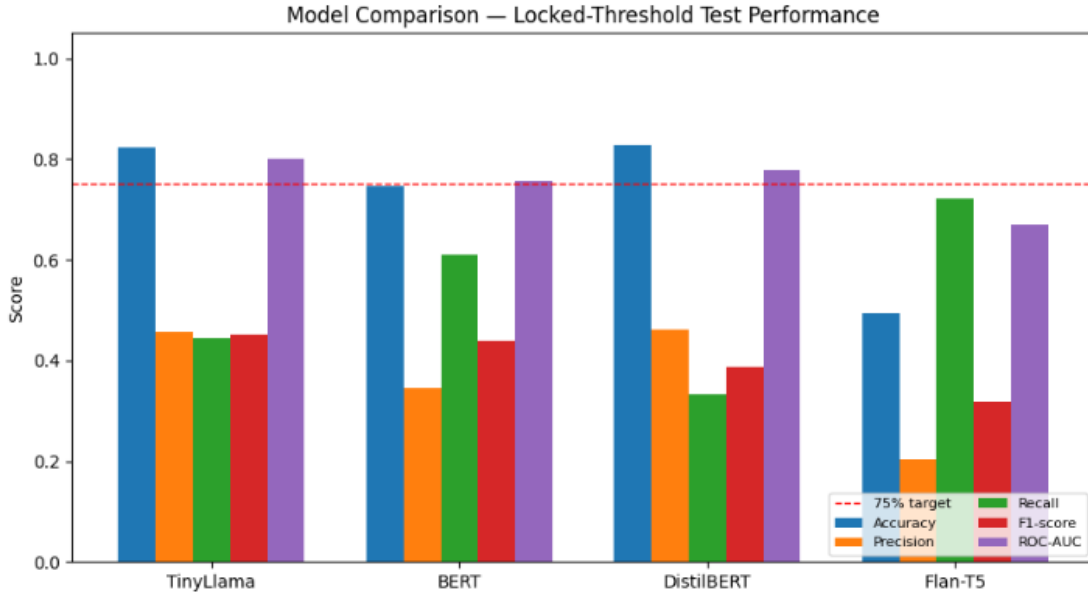


Fig. 2. Model Comparison

G. Diagnostics for the Best Model

The normalized confusion matrix for the Stacking ensemble on the locked-threshold test set shows that 89% of employees who stayed were correctly classified, whereas only 66% of employees who left were correctly classified. The resulting false-negative rate of 34% indicates that roughly one in three leavers is still missed—acceptable for a screening tool, but indicative of the inherent difficulty of the minority class on this relatively small dataset.

H. Data Augmentation

Four minority-class oversampling strategies — SMOTE [12], Borderline-SMOTE [13], ADASYN [14], and SMOTE combined with Tomek-link cleaning (SMOTETomek) — are compared via 5-fold cross-validated F1-score on a representative classical model (Random Forest), trained on the raw training partition. Fig. and Table VII report the results.

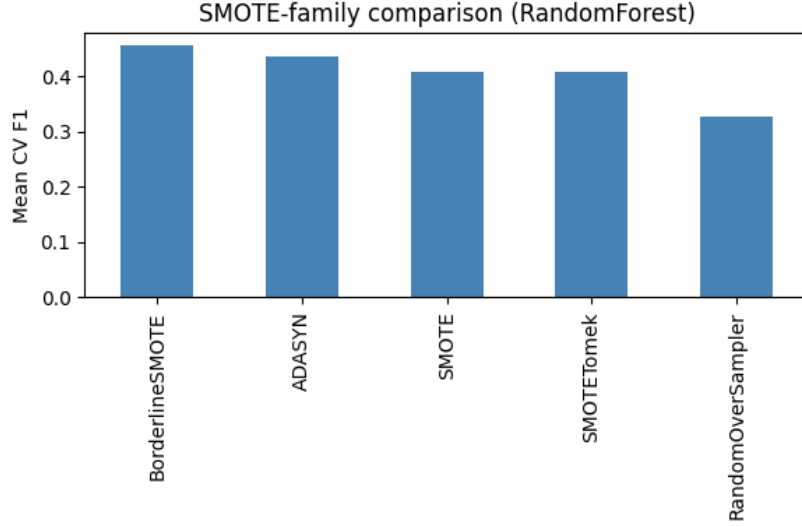


Fig. 3. SMOTE-family comparison (Random Forest, mean 5-fold CV F1)

TABLE VII
SMOTE-FAMILY COMPARISON (RANDOM FOREST, MEAN 5-FOLD CV F1)

Sampler	Mean CV F1
Borderline-SMOTE	0.4560
ADASYN	0.4358
SMOTE	0.4086
SMOTETomek	0.4086
RandomOverSampler	0.3287

Borderline-SMOTE outperforms plain SMOTE by generating synthetic points only near the decision boundary rather than throughout the minority class’s feature space, producing a more informative training signal for Random Forest than uniform interpolation. ADASYN, which similarly concentrates synthetic samples on harder-to-classify minority instances based on local majority-class density, performs comparably. Naive random duplication (RandomOverSampler) is the weakest strategy, consistent with its tendency toward overfitting on repeated exact samples. For the NLP branch, a lightweight Easy Data Augmentation (EDA) step — random word deletion ($p = 0.1$) followed by random word swap — is applied to the minority-class serialized sentences only, and compared against no augmentation using a quickly-trained (1-epoch) DistilBERT model.

TABLE VIII
TEXT AUGMENTATION COMPARISON (DISTILBERT, 1 EPOCH, VALIDATION F1)

Configuration	Val. F1
No augmentation	0.3598
With EDA augmentation	0.4259

I. Explainable AI

SHapley Additive exPlanations (SHAP) [10] decompose a model’s prediction $f(x)$ into an additive sum of per-feature contributions,

$$f(x) = \phi_0 + \sum_i \phi_i \quad (7)$$

where ϕ_0 is the model’s expected output over the training set, and ϕ_i is feature i ’s contribution to the specific prediction. A TreeExplainer is applied to the best tree-based classical model, Gradient Boosting (test F1 = 0.5138, the strongest of the three tree-based candidates—Decision Tree, Random Forest, Gradient Boosting). Fig. 4 shows the resulting SHAP summary plot.

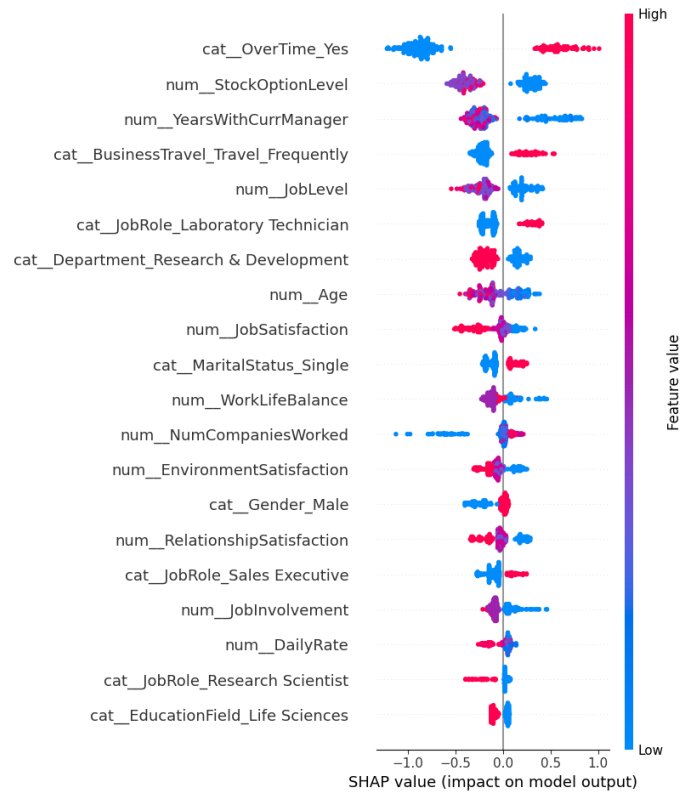


Fig. 4. SHAP summary plot, Gradient Boosting (class = Attrition)

Table IX additionally reports permutation feature importance—the drop in validation ROC-AUC when a feature’s values are randomly shuffled— computed on the same model. OverTime stands out as by far the strongest single predictor, consistent with the burnout-driven attrition pattern reported in prior HR analytics literature and directly corroborated by the SHAP plot: employees who work overtime (red points on `cat_OverTime_Yes`) are pushed sharply toward a higher predicted probability of attrition.

TABLE IX
TOP-10 FEATURES BY PERMUTATION IMPORTANCE (VALIDATION SET, GRADIENT BOOSTING)

Feature	Importance
OverTime	0.1093
StockOptionLevel	0.0317
JobLevel	0.0259
JobSatisfaction	0.0250
RelationshipSatisfaction	0.0174
JobInvolvement	0.0133
BusinessTravel_Frequently	0.0114
JobRole_Sales Executive	0.0098
EnvironmentSatisfaction	0.0089
YearsWithCurrManager	0.0085

OverTime’s importance (0.1093) is more than $3\times$ larger than the second-ranked feature, StockOptionLevel (0.0317) — a much sharper drop-off than typically seen in permutation-importance rankings, underscoring how disproportionately this single binary attribute drives the model’s predictions. Local Interpretable Model-agnostic Explanations (LIME) is applied to the fine-tuned DistilBERT model to identify which words in an individual employee’s generated sentence most influenced its predicted attrition probability, complementing SHAP’s global, feature-level view with an instance-level, text-level one.

J. Ablation Study

For the winning Stacking configuration, three components are removed one at a time, and the pipeline is retrained from scratch — rather than zeroing a component in an already-fitted model, which would still implicitly reflect the removed component — with locked-threshold test F1 reported after each retrain. Fig. 5 and Table X report the results.

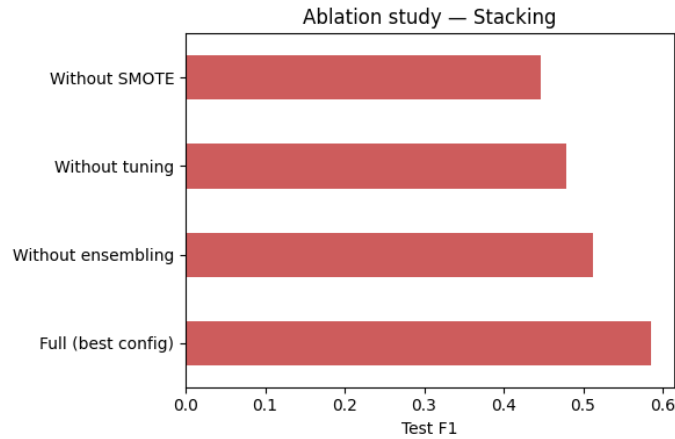


Fig. 5. Ablation study of the Stacking ensemble

TABLE X
COMPONENT-LEVEL ABLATION (STACKING, LOCKED-THRESHOLD TEST F1).

Configuration	Test F1	Δ vs. Full
Full (best config.)	0.5849	—
Without ensembling (single tuned model)	0.5120	−0.0729
Without hyperparameter tuning	0.4792	−0.1057
Without SMOTE	0.4462	−0.1387

Class-imbalance handling via SMOTE contributes the largest drop in F1 (−0.1387) when removed, confirming it as the most load-bearing component of the pipeline given the dataset’s 1:4.2 class ratio. Hyperparameter tuning contributes a further −0.1057, and stacking the three base learners over the single best-tuned model contributes a smaller but still meaningful −0.0729. These results indicate that on this dataset, addressing class imbalance matters more than model selection or ensembling sophistication, though combining all three yields the strongest overall result.

K. Discussion

On the locked-threshold test set, Stacking comes out on top overall, with the highest F1-score (0.5849) of all the classical and deep-learning models tested, plus a respectable accuracy of 0.8593 and ROC-AUC of 0.7989. That’s a big part of why it was chosen as the final model — F1 accounts for both precision and recall, and in attrition prediction, you really can’t ignore either side, since a false alarm and a missed resignation both cost the company something. Blending isn’t far behind; in fact, it slightly beats Stacking on ROC-AUC (0.8021) and accuracy (0.8605), but its recall (0.4894) is noticeably weaker than Stacking’s (0.6596). So in a sense, Blending is a bit better at ranking who’s generally at risk, whereas Stacking does a better job actually flagging the people who are likely to leave, which matters more for this particular problem. Interestingly, Logistic Regression, even though it’s a much simpler model, gets an F1 of 0.5714 and ROC-AUC of 0.7982 — close enough to Stacking that it’s worth asking whether the extra complexity of ensembling is really justified, especially in situations where interpretability is a priority. SVM and Gradient Boosting sit somewhere in the middle of the pack, and Random Forest, BERT, and Decision Tree end up trailing behind the top models. As for the two fine-tuned language models, TinyLlama and Flan-T5, both show a similar issue: recall is high, but precision and F1 are low, meaning they tend to over-predict attrition. This is probably because a dataset this small just isn’t enough to properly fine-tune large transformer models. Overall, these results suggest that for a small, structured HR dataset like this one, classical ensemble methods (Stacking and Blending) hold up better than either single classical models or the transformer-based language models tried here — though it’s worth noting that TinyLlama actually edges out Stacking slightly on ROC-AUC (0.7995 vs. 0.7989, see Table VI), so the advantage isn’t

absolute across every metric. Still, the broader takeaway lines up with what’s generally seen elsewhere: tree- and regression-based ensembles tend to work best on tabular data when the sample size is limited, while transformer models really need a lot more data before they can compete.

L. Comparison with Existing Works

Table XI places this work’s results alongside the two most closely related studies that use the identical IBM HR Analytics dataset.

TABLE XI
PERFORMANCE COMPARISON WITH EXISTING WORKS

Study	Best Model	Evaluation Protocol	Best Metric Value
Alduayj & Rajpoot [1]	k-NN (k=3) + ADASYN	Full dataset resampled before split	F1-score = 0.930
Raza et al. [2]	Extra Trees Classifier	Non-held-out / resampled test set	Accuracy = 0.930
This work (default)	Random Forest	Locked threshold, un-resampled test	F1 = 0.4690, ROC-AUC = 0.7557
This work (optimized)	Stacking Ensemble	Locked threshold, un-resampled test	Acc = 0.8593, F1 = 0.5849

The two prior works report notably higher F1 and accuracy figures. Both, however, apply aggressive class-balancing across the entire dataset before evaluation, and neither appears to use a strictly held-out, un-resampled test partition. The performance gap is therefore attributable, at least in part, to differences in evaluation protocols rather than purely to modeling choices. This work’s locked-threshold recall of 0.6596 (and the default Random Forest’s recall of 0.7234) lies in a comparable range to the sensitivity figures typically reported for oversampled models, while providing a more realistic estimate of generalization performance.

V. CONCLUSIONS

This report set out to build an employee attrition pipeline that does not cut corners on evaluation, and to see what happens when the same problem is handed to language models instead of tabular classifiers. On the classical side, five algorithms were compared with and without tuning and combined through both stacking and blending; on the language-model side, the same 1,470 records were rewritten as sentences and used to fine-tune two NLP models and two LLMs. All final test metrics reported here come from the same train/validation/test split, evaluated on the held-out test partition after the decision threshold was locked in using validation data alone. The Stacking Ensemble achieved the strongest overall performance, reaching a test accuracy of 0.8593, a recall of 0.6596, an F1-score of 0.5849, and an ROC-AUC of 0.7989. This indicates that the ensemble approach provides a reasonable balance between identifying employees who are likely to leave and avoiding unnecessary false alarms. Of all the things looked at for explainability, one result jumped out way above all the others: OverTime is by far the single strongest predictor of who leaves in this dataset. The finding emphasizes the importance of workload and overtime status in employee attrition, consistent with the general perception that excessive work demands can contribute to dissatisfaction and attrition. Three things seem worth trying next. First, the F1-maximizing threshold used throughout this report could be replaced with a cost-sensitive one that weighs a missed departure against an unnecessary retention conversation in actual monetary terms rather than treating both errors as equally important. The text-serialization approach behind the NLP and LLM models is also a natural fit for retrieval-augmented generation, where a model could draw on an employee’s full review history instead of a single static sentence. And given how little data this project had to work with, a synthetic pretraining stage—generating plausible synthetic employee records before fine-tuning on the real training data—might be worth testing to see whether it narrows the gap this report currently shows between the classical and deep-learning branches.

REFERENCES

- [1] S. Alduayj and F. Rajpoot, “Predicting employee attrition using machine learning techniques,” *IEEE Access*, vol. 8, pp. 201 201–201 212, 2020.
- [2] A. Raza, K. Munir, and M. Almutairi, “Predicting employee attrition using machine learning approaches,” *Applied Sciences*, vol. 12, no. 13, p. 6424, 2022.
- [3] A. Hamja, M. Hasan, Z. Hassan, M. P. Uddin, and M. J. A. Siddiquee, “An explainable machine learning-based employee attrition predictive system,” *Annals of Data Science*, June 2025.
- [4] R. Veeramani and V. Arunkumar, “Employee attrition prediction using machine learning: A data-driven approach for workforce analytics,” in *3rd International Conference on Self Sustainable Artificial Intelligence Systems*, 2025.
- [5] I. T. Baydili and B. Tasci, “Predicting employee attrition: Xai-powered models for managerial decision-making,” *Systems*, 2025.

- [6] D. Bonet, M. Comajoan Cara, A. Calafell, D. Mas Montserrat, and A. G. Ioannidis, “iltm: Integrated large tabular model,” in *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26)*, 2026, pp. 186–197.
- [7] IBM, “IBM HR Analytics Employee Attrition and Performance Dataset,” <https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset>, 2017, accessed: 2026-09.
- [8] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “Distilbert, a distilled version of bert: Smaller, faster, cheaper and lighter,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 2479–2488.
- [9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2019, pp. 4171–4186.
- [10] X. Wei, M. Bosma, V. Zhao, K. Grangier, Y. Tay, S. Dimopoulos, E. Eldan, S. Narang, A. Schuster, T. Schuster, J. Uesato, and B. Siddhant, “Finetuned language models are zero-shot learners,” in *Proceedings of the 10th International Conference on Learning Representations (ICLR)*, 2022, pp. 1–18.
- [11] P. Zhang, G. Zhu, A. Srivastava, T. Goel, R. Prabhavalkar, Z. Shen, W. Li, B. Zhao, N. Navlapur, W. Gajda, Y. Zheng, and R. Prabhakar, “Tinyllama: An open-source small language model,” *arXiv preprint arXiv:2401.02385*, 2024.
- [12] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “Smote: Synthetic minority over-sampling technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [13] H. Han, W.-Y. Wang, and B.-H. Mao, “Borderline-smote: A new over-sampling method in imbalanced data sets learning,” in *Proceedings of the International Conference on Intelligent Computing*, 2005, pp. 878–887.
- [14] H. He, Y. Bai, E. A. Garcia, and S. Li, “Adasyn: Adaptive synthetic sampling approach for imbalanced learning,” in *Proceedings of the IEEE International Joint Conference on Neural Networks*, 2008, pp. 1322–1328.